ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 3

INTRODUCTION TO TECHNICAL PROGRAMMING

PAPER 2 · PRACTICE E

MARKS: 100 **TIME: 3 hours**

NC(V) Programming and Robotics, Level 3. Laid out like the DHET second paper (practical, 100 marks): an mBlock electricity-token estimator, a C++ temperature converter with categories, and a C++ running-club program with arrays, each with an IPO chart.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 3. It is not a Department of Higher Education and Training examination paper.

INSTRUCTIONS AND INFORMATION

1. This question paper consists of THREE questions. Answer ALL the questions.
2. This is a practical paper. Use mBlock for QUESTION 1 and a C++ development environment (such as Geany or Code::Blocks) for QUESTIONS 2 and 3. If you do not have a computer, write the programs neatly in your answer book.
3. Save each program with the question number and your name or examination number, and add your name or examination number as a comment on the first line of each C++ program.
4. Save your work regularly. Programs are not marked strictly on syntax.

QUESTION 1

A family buys prepaid electricity. One unit (kWh) costs R2,50. The family wants an mBlock app that asks how much money they will spend and for how many days the electricity must last, then works out the number of units and the units available per day, and gives advice: fewer than 10 units per day – "Careful, save electricity"; 10 to 20 units per day – "Good"; more than 20 units per day – "Plenty".

INPUT	PROCESS	OUTPUT
Amount (R) Number of days	Declare all variables and set them to 0 Ask for the amount and the number of days units = amount ÷ 2.5 perDay = units ÷ days If perDay < 10: "Careful, save electricity" Else if perDay ≤ 20: "Good" Else: "Plenty"	units perDay advice

Follow the instructions below to create the mBlock application:

1.1 Open mBlock, create a new project and name it Question1_YourName. (1)

1.2 Choose a suitable sprite. (1)

1.3 Choose a backdrop that shows a home or a kitchen. (1)

1.4 Declare all the variables and give them their starting values. (8)

1.5 Ask for the amount of money and store the answer. (3)

1.6 Ask for the number of days and store the answer. (3)

1.7 Calculate the units and the units per day. (4)

1.8 Use if … then … else blocks to choose the advice. (6)

1.9 Display the units, the units per day and the advice. (3)

1.10 Test the program with R500 for 30 days, R1 000 for 30 days and R750 for 10 days, and write down what the sprite says. (2)

[32]

QUESTION 2

A weather station wants a C++ program that changes a temperature in degrees Celsius into degrees Fahrenheit and kelvin, and shows a category: below 0 °C – "Freezing"; 0 °C up to and including 25 °C – "Mild"; above 25 °C – "Hot".

Formulas: Fahrenheit = Celsius × 9 ÷ 5 + 32 and Kelvin = Celsius + 273.15

INPUT	PROCESS	OUTPUT
Temperature in °C	Declare all variables and set them to 0 Ask for the temperature Calculate Fahrenheit and Kelvin If C < 0: "Freezing"; else if C ≤ 25: "Mild"; else "Hot"	Fahrenheit Kelvin category

2.1 Create a new C++ source file called Question2_YourName.cpp. (1)

2.2 Include the necessary header(s) and use the std namespace. (2)

2.3 Define the main function and declare all variables, setting them to 0 (or an empty string). (4)

2.4 Ask the user for the temperature in °C and store it. (2)

2.5 Calculate the temperature in Fahrenheit and in kelvin. (5)

2.6 Use if, else if and else to find the category. (8)

2.7 Display the temperature in Fahrenheit and kelvin, and the category. (3)

[25]

QUESTION 3

A running club times its members over 5 km. The coach wants a C++ program that stores the times (in minutes) of up to 20 runners in an array and then shows the average time, the fastest and slowest times, the number of runners who finished in under 30 minutes, and every time that was slower than the average.

INPUT	PROCESS	OUTPUT
Number of runners Time of each runner	Declare all variables Ask for the number of runners until it is from 1 to 20 Ask for each time until it is more than 0, and store it in an array Calculate the total and the average Find the fastest and slowest times Count the times under 30 minutes List the times slower than the average	Average Fastest and slowest Number under 30 min Times slower than average

The following steps show how the program should be used:

```
How many runners (1-20)? 4
Time for runner 1 (min): 28.5
Time for runner 2 (min): 0
Time must be more than 0.
```

```
Time for runner 2 (min): 35
Time for runner 3 (min): 31.5
Time for runner 4 (min): 25
Average time: 30 min
Fastest: 25 min   Slowest: 35 min
Under 30 minutes: 2 runners
Slower than average: 35 31.5
```

3.1	Create a new C++ source file called Question3_YourName.cpp.	(1)
3.2	Include the necessary header(s) and use the std namespace.	(2)
3.3	Define the main function and declare the variables.	(4)
3.4	Ask for the number of runners, and ask again until the number is from 1 to 20.	(4)
3.5	Declare the array that stores the times.	(3)
3.6	Use a loop to ask for each time, check that it is more than 0 (ask again if it is not) and store it in the array.	(8)
3.7	Calculate the total and the average time.	(4)
3.8	Find the fastest and the slowest times.	(8)
3.9	Count the runners who finished in under 30 minutes, and display every time that is slower than the average.	(4)
3.10	Display the average, the fastest and slowest times and the number of runners under 30 minutes.	(5)

[43]

TOTAL: 100

END OF QUESTION PAPER